Design and Development of a Retrieval-Augmented Generation-Based Persian Text Humanization Chatbot Using Large Language Models

Rizwanullah Muahed
Department of Computer Science
Kabul Polytechnic University
Kabul, Afghanistan
rizwanullahmuahed2@gmail.com

Abdul Awal Quraishi
Department of Computer Science
Kabul Polytechnic University
Kabul, Afghanistan

***Abstract*—** AI-generated Persian (Dari) text is often grammatically correct but still reads as AI-generated – stiff phrasing, redundant precision, formulaic sentence patterns – regardless of whether the underlying content is formal, casual, technical, or conversational. This paper addresses that broader problem, humanization, rather than style transfer between registers: the goal is to rewrite AI-like Persian into natural, human-sounding text while preserving its meaning, facts, and original tone, treating the input's register as a characteristic to be respected rather than a target to convert toward. We build and describe a Retrieval-Augmented Generation (RAG) chatbot that retrieves semantically similar example rewrites from a curated Persian parallel corpus and folds them into the prompt given to a locally hosted Llama 3.1 8B model, coordinated through a modular pipeline (multilingual sentence embeddings, ChromaDB for a single-pass similarity search, LangGraph for orchestration, FastAPI for the backend, and Ollama for fully offline inference). We assembled a parallel corpus of 3,173 AI-generated/human-written Persian sentence pairs; each pair also carries register and domain metadata (ten register categories, eight recorded register transitions) that supports retrieval analysis but is not itself the transformation target. Qualitative inspection of the generated output shows edits consistent with genuine humanization – simplified verb constructions, reduced bureaucratic phrasing, softened register – while numeric details, obligations, and factual content are preserved. We report retrieval precision/recall, ROUGE (interpreted in the reverse of its usual direction, since low overlap here indicates rewriting rather than copying), and output-length stability as characterizations of the assembled system, and we are explicit that these are not a substitute for the comparison the paper's central claim actually requires: an LLM-only baseline against LLM+RAG, and a comparison across retrieval depths (e.g., top-1 vs. the deployed default of top-4). That comparison, along with a held-out retrieval evaluation that excludes self-retrieval, has not yet been run and is identified as the paper's most important direction for follow-up work rather than a claim made here. The current system also does not include style-based retrieval filtering or a reranking stage; retrieval is a single embedding-similarity search, and we discuss filtering/reranking only as candidate future extensions. We see the paper's contribution as distributed across a working, fine-tuning-free architecture, the accompanying dataset, and an evaluation methodology honest about what has and has not yet been measured.

***Index Terms*—** Artificial Intelligence, Natural Language Processing, Large Language Models, Retrieval-Augmented Generation, Persian Text Humanization, LangGraph, ChromaDB, FastAPI

I. Introduction

Large language models have reshaped what's possible in natural language processing over the past few years, pushing the state of the art in machine translation, summarization, question answering, and open-ended content generation. Persian has been slower to benefit. The language's distinctive formal/informal register split, its flexible word order, and the comparative scarcity of large, high-quality Persian corpora all make fluent, natural-sounding Persian generation harder to achieve than the equivalent task in English or other high-resource languages.

That gap shows up in practice regardless of register: AI writing assistants are increasingly expected to produce text that reads as though a person wrote it, not a system. Current language models generate Persian that is grammatically sound but still often carries recognizably AI-like patterns – redundant precision, formulaic sentence rhythm, nominalized bureaucratic constructions, hedged phrasing – whether the underlying text is meant to be formal, casual, technical, or conversational. This is a broader problem than formality alone: a casual AI-generated message can read as stilted in the same way a formal one can, and the fix in either case is not to change which register the text uses but to make the chosen register sound genuinely human. This paper refers to that broader problem as humanization, and treats register (formal, casual, technical, and so on) as a property of the input to be respected during rewriting, not as the thing being optimized.

Recent research has shown that Retrieval-Augmented Generation (RAG) improves language generation by enriching the prompting process with semantically relevant examples retrieved from external knowledge sources [1]–[3]. Instead of depending exclusively on the internal parameters of a language model, RAG retrieves contextually similar information before generation, enabling more accurate and context-aware responses. At the same time, the emergence of in-context learning has demonstrated that Large Language Models can perform complex language tasks effectively without repeated model fine-tuning [4], [5]. These developments provide an efficient foundation for designing practical Persian language applications.

Several previous studies have investigated Persian text style transfer, formality transformation, and text normalization [5]–[7]. This existing work is closely related but narrower in scope than the problem addressed here: it targets a specific register transition (typically informal-to-formal or the reverse) as the object of study, whereas this paper treats register as something to be preserved and targets the more general symptom of AI-like phrasing regardless of register. Most of this prior work also relies on supervised learning and fine-tuned language models that require large annotated datasets, repeated training, and substantial computational resources – requirements that increase development complexity and reduce accessibility for educational institutions and individual developers, independent of the style-transfer-versus-humanization distinction.

To address these limitations, this paper investigates a specific question: can retrieval-augmented, in-context prompting – without any fine-tuning – make AI-generated Persian text read as natural, human-written text, while preserving its meaning, facts, and original register? Rather than modifying the language model's parameters, the proposed system retrieves semantically similar AI-generated/human-written text pairs from a curated parallel dataset and incorporates them into the prompt to guide text generation. Semantic retrieval is performed using multilingual sentence embeddings and ChromaDB, while LangGraph manages the complete workflow from retrieval to response generation. The language-generation component is implemented using the Llama 3.1 8B model deployed locally through Ollama, enabling the entire system to operate without cloud-based AI services.

The proposed architecture adopts a modular design consisting of separate layers for data management, semantic retrieval, workflow orchestration, language generation, backend communication, and user interaction. This modular structure improves maintainability, simplifies future upgrades, and allows individual components to be replaced independently without affecting the overall architecture.

This paper's contributions span four distinct kinds of work, which we separate deliberately rather than presenting as a single undifferentiated list, and which we do not conflate with each other – in particular, integrating existing tools such as FastAPI, LangGraph, and ChromaDB is engineering work, not a research finding, and is labeled as such below:

- Research contribution: qualitative evidence, drawn from inspection of generated output, that retrieval-augmented in-context prompting can produce consistent, linguistically interpretable humanization edits – reduced bureaucratic phrasing, simplified verb constructions, softened register – without altering propositional content or the input's original register. We describe this as preliminary: a controlled comparison against a no-retrieval baseline, which would isolate retrieval's causal contribution, is identified as the paper's most important open question (Section IV-F) rather than a claim settled here.
- Dataset contribution: a Persian parallel corpus of 3,173 AI-generated/human-written sentence pairs, each additionally labeled with register and domain metadata (ten register categories, eight recorded register transitions) that supports retrieval and post-hoc analysis without being the primary training or evaluation target.
- Engineering contribution: a modular, fine-tuning-free RAG architecture – multilingual sentence embeddings, ChromaDB, LangGraph, FastAPI, Ollama, and Llama 3.1 8B integrated into a single offline pipeline – that is straightforward to reproduce, extend, or swap components within.
- Evaluation contribution: a reframing of overlap-based metrics for this task – since humanization is expected to reword rather than reproduce its input, we argue (and empirically confirm on our data) that a low ROUGE score, not a high one, is the signal of successful rewriting – paired with an explicit account of what the current evaluation does not yet establish, including a concrete experimental design (no-retrieval baseline, held-out retrieval evaluation, retrieval-depth comparison) for the follow-up work needed to confirm the research contribution above.

The remainder of this paper is organized as follows. Section II reviews related studies on Persian text generation, style transfer, and Retrieval-Augmented Generation. Section III describes the proposed methodology and system architecture. Section IV presents the implementation details and experimental results. Finally, Section V concludes the paper and discusses future research directions.

II. Related Work

Recent research in Natural Language Processing (NLP) has significantly advanced text generation, text style transfer, and Retrieval-Augmented Generation (RAG). These developments have created new opportunities for generating fluent and context-aware text using Large Language Models (LLMs). However, research specifically focused on Persian text humanization remains limited, particularly in approaches that combine semantic retrieval with prompt-based language generation.

Text style transfer (TST), the broader task family to which Persian text humanization belongs, has evolved considerably over the past decade. Early neural approaches relied on the sequence-to-sequence (seq2seq) architecture [8], which compresses an input sentence into a fixed vector before decoding it

into a new form; this design, however, suffers from information loss on longer inputs and treats all source tokens as equally important. To reduce their dependence on large parallel corpora, several works reframed style transfer without requiring paired supervision: the Delete-Retrieve-Generate framework [9] separates style-bearing segments from content and substitutes them individually, while back-translation approaches [10] disentangle style and content implicitly by translating a sentence into an intermediate language and back, injecting the target style during the return translation. These techniques illustrate a broader trend in the TST literature: substituting expensive parallel supervision with retrieval, substitution, or self-supervised proxies – a philosophy that directly motivates the retrieval-based, fine-tuning-free approach adopted in this work.

One of the most relevant studies is Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1], which introduced the RAG framework by combining neural retrieval with sequence generation models. The study demonstrated that retrieving relevant contextual information before text generation improves factual consistency and response quality. This work established the theoretical foundation for many modern retrieval-enhanced language generation systems.

The emergence of Transformer-based language models further accelerated progress in text generation. Vaswani et al. [11] introduced the Transformer architecture, which replaced recurrent neural networks with self-attention mechanisms and became the foundation of modern LLMs. Later, Brown et al. [4] demonstrated that large language models are capable of solving diverse language tasks through in-context learning, reducing the dependence on task-specific model training.

Recent open-source language models have also contributed to practical AI system development. The Llama 3 model family [12] provides strong multilingual language generation capabilities while allowing local deployment through frameworks such as Ollama. Local execution improves data privacy, reduces operational costs, and enables offline AI applications without relying on commercial cloud services.

Alongside advances in generation, dense retrieval methods such as Dense Passage Retrieval [13] have shown that learned vector representations can outperform traditional sparse retrieval for selecting relevant context, forming the conceptual retrieval backbone that RAG-based systems, including the one proposed here, build upon. Complementary work on neural passage re-ranking [14] has further shown that a lightweight re-ranking stage over an initial retrieved set can improve the relevance of the context passed to a downstream language model; we discuss this line of work as motivation for future extensions to the retrieval module (Section IV-F), since the system described here performs a single embedding-similarity search and does not currently include a re-ranking stage.

Several studies have specifically investigated Persian language processing. Tajalli et al. [7] introduced a parallel Persian corpus for informal-to-formal text transformation, providing an important resource for Persian style transfer research. Building on a similar resource, Falakaflaki and Shamsfard [6] proposed Fa-BERT2BERT, a Transformer-based model for Persian formality style transfer trained with supervised consistency learning; notably, they also showed that conventional overlap-based metrics such as BLEU and ROUGE can be misleading for this task, since a model that merely copies its input can still score deceptively well when many words share identical formal and informal forms – an observation that directly informs how this paper interprets its own ROUGE results in Section IV. Separately, Rasooli et al. [15] addressed Persian text normalization by framing standardization of colloquial Persian as a sequence-to-sequence translation problem, using synthetically generated parallel data to convert informal text into its standard written form. Although these methods achieved encouraging results, they primarily focus on supervised training and require large annotated datasets together with computationally expensive fine-tuning.

For representing Persian text itself, monolingual pretrained encoders such as ParsBERT [16] have demonstrated strong performance on Persian-specific NLP tasks. In contrast, the proposed system relies on multilingual sentence embeddings for retrieval, trading some language-specific optimization for the flexibility of a shared multilingual vector space.

More recently, several survey papers have highlighted the advantages of Retrieval-Augmented Generation for Large Language Models [2], [3]. These studies conclude that semantic retrieval improves contextual relevance, reduces hallucination, and enables language models to generate more accurate responses without modifying the underlying model parameters. These characteristics make RAG particularly suitable for languages with relatively limited training resources, including Persian.

Despite these advances, an important research gap remains. Most existing Persian studies emphasize text normalization or formality transformation, whereas relatively little attention has been given to humanizing AI-generated Persian text using Retrieval-Augmented Generation. Furthermore, many existing approaches rely on cloud-based AI services or computationally expensive fine-tuning, limiting their practical deployment in educational and research environments.

To address these limitations, the present study proposes a modular Persian text humanization chatbot based on Retrieval-Augmented Generation. The proposed architecture combines multilingual sentence embeddings, ChromaDB, LangGraph, FastAPI, and a locally deployed Llama 3.1 8B language model to generate more natural Persian text while preserving semantic meaning. Unlike previous Persian approaches, the proposed system performs semantic retrieval during inference instead of retraining the language model, providing a lightweight, privacy-preserving, and extensible solution.

Table I compares the proposed system with representative studies discussed in this section.

TABLE I
Comparison of Previous Studies

Study	Persian Support	RAG	Fine-tuning Required	Primary Objective
Lewis et al. [1]	×	✓	×	Retrieval-Augmented Generation
Brown et al. [4]	General	×	×	In-context Learning
Tajalli et al. [7]	✓	×	×	Persian Parallel Corpus
Falakaflaki & Shamsfard [6]	✓	×	✓	Persian Formality Style Transfer
Rasooli et al. [15]	✓	×	✓	Persian Text Normalization
Proposed System	✓	✓	×	Persian Text Humanization

III. Proposed Methodology

This section describes the methodology adopted for designing and implementing the proposed Persian text humanization chatbot. The system is built upon a Retrieval-Augmented Generation (RAG) architecture that combines semantic retrieval with a Large Language Model (LLM) to generate more natural Persian text while preserving the semantic meaning of the original input. Instead of modifying

the language model through fine-tuning, the proposed approach retrieves semantically relevant examples from a Persian parallel dataset and uses them to guide text generation.

The proposed methodology consists of five main components: the overall system architecture, the Persian parallel dataset, the semantic retrieval module, the language generation workflow, and the backend implementation.

A. Overall System Architecture

The proposed chatbot follows a modular architecture composed of four logical layers: the Data Layer, Retrieval Layer, Processing Layer, and Presentation Layer. This layered design separates the responsibilities of each component, improving maintainability and simplifying future system extensions.

The Data Layer stores the Persian parallel dataset together with the vector database. The Retrieval Layer performs semantic search by converting input text into vector embeddings and retrieving the most relevant examples from ChromaDB. The Processing Layer executes the Retrieval-Augmented Generation workflow through LangGraph and communicates with the Large Language Model to generate the final response. Finally, the Presentation Layer provides interaction between users and the backend services through a web-based interface.

Fig. 1 illustrates the overall architecture of the proposed Persian text humanization chatbot. The diagram groups components into the four layers introduced above and traces the runtime data flow across them (numbered steps 1–6): a request enters through the Presentation Layer and is forwarded to the Processing Layer's LangGraph orchestrator, which passes the original input to the Retrieval Layer; there it is embedded and matched against the Data Layer's ChromaDB index to obtain the top-k examples. As highlighted in the figure, the retrieved examples and the original user input are both routed into Prompt Construction, where they are combined into a single prompt before generation – the point at which retrieval and the original input actually merge, prior to being sent to the Llama 3.1 8B model.

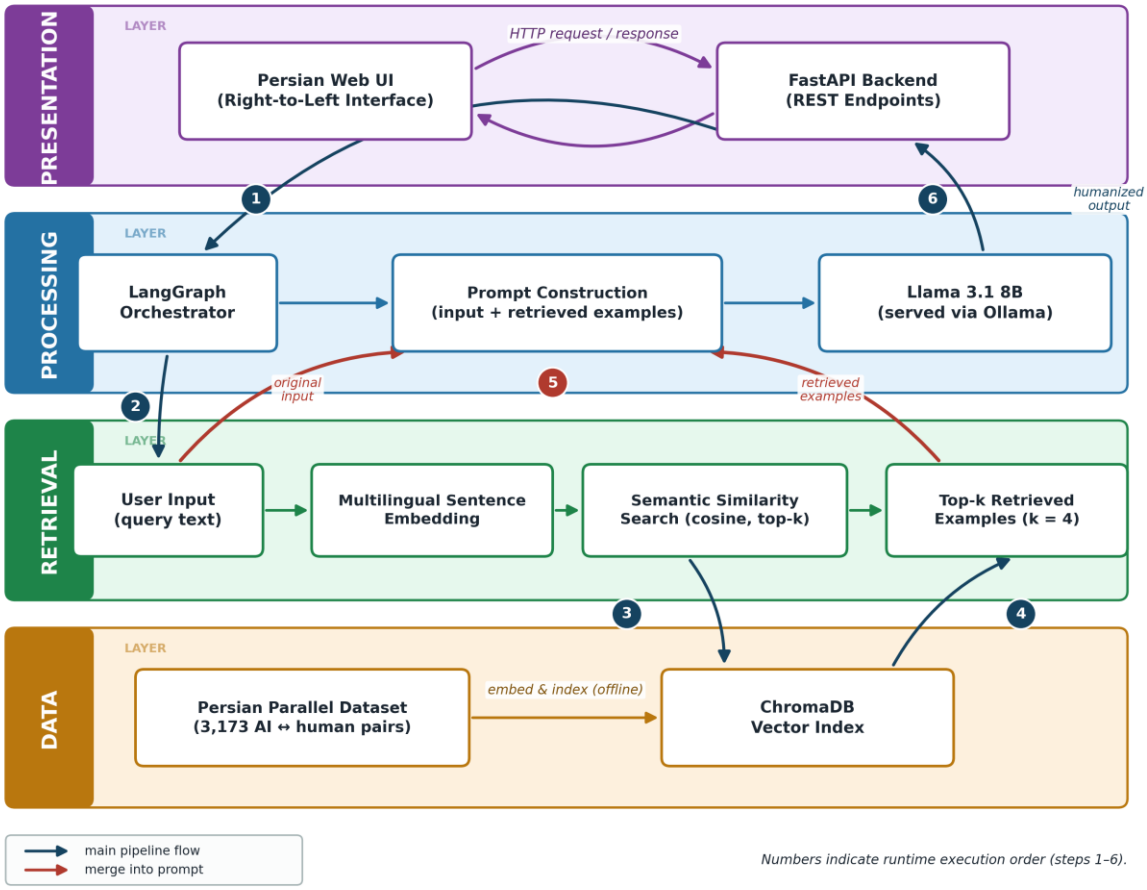


Fig. 1. System architecture of the proposed Persian text humanization chatbot. Components are grouped into the four layers described in Section III-A (Data, Retrieval, Processing, Presentation); numbered arrows trace the runtime execution order, and the highlighted arrows show retrieved examples and the original user input converging into Prompt Construction before generation.

B. Persian Parallel Dataset

To support semantic retrieval, a Persian parallel dataset was prepared whose core pairing is an AI-generated (or AI-like) Persian input matched with a natural, human-written rewrite that preserves the same meaning. Each record additionally carries register and domain metadata – a style label and a recorded register transition – describing the pair, but this metadata is not the transformation target: the system's objective is to produce natural, human-sounding text appropriate to each input's own register, not to move text toward any particular register. The metadata exists to support retrieval analysis (Section IV) and to let the corpus be filtered or audited by register, not to define what "success" means for a given rewrite. Before indexing, the dataset was reviewed, cleaned, and organized to improve retrieval quality and ensure consistency among the recorded registers.

In its final form, the dataset comprises 3,173 AI-generated/human-written text pairs. Each pair is labeled with one of ten register categories and one of eight recorded register transitions (e.g., formal_to_casual); these labels describe the composition of the source data collected for the corpus rather than a target distribution the system is designed to produce, and their composition is summarized quantitatively in Fig. 4 (Section IV).

The dataset serves as the primary knowledge source for the Retrieval-Augmented Generation process. During inference, semantically similar examples are retrieved from this dataset and

incorporated into the language model prompt; retrieval is based purely on semantic similarity between the user's input and the indexed AI-generated texts, without filtering or biasing by the register metadata.

TABLE II
Structure of the Persian Parallel Dataset

Column	Explanation	Example
Ai	AI-generated/AI-like input text	*"This request is being reviewed"*
Human	Natural, human-written rewrite	*"We will review your request"*
Style	Register metadata (not the target)	casual, friendly, …
Transformation	Recorded register shift (metadata)	formal_to_casual

C. Semantic Retrieval Module

The semantic retrieval module is responsible for identifying examples that are most similar to the user's input, following the dense retrieval paradigm [13]. First, the input Persian text is transformed into a dense vector representation using a multilingual sentence embedding model [17], itself an extension of the Sentence-BERT architecture [18] to a shared multilingual space via knowledge distillation. The embedding process can be represented as

$$q = f(x)$$

where x denotes the input text, f(·) represents the embedding model, and q is the semantic vector generated for retrieval.

The generated embedding is compared with vectors stored in ChromaDB through cosine-similarity search, and the top-k closest matches are returned in similarity order. In the deployed system, k = 4 is used as a fixed default (Fig. 1); this value was not tuned against an evaluation metric and is not claimed to be optimal. It is important to be precise about what this step does and does not do: retrieval consists of a single embedding-similarity search, with no separate re-ranking model applied to reorder the candidates afterward, and no filtering by the register metadata described in Section III-B – a retrieved example's register match to the query is an emergent consequence of semantic similarity, not an explicit retrieval constraint. Adding a re-ranking stage or register-aware filtering is discussed as a candidate future extension in Section IV-F, not as a component of the current system. The retrieved examples are prepared for prompt construction in this similarity order; this retrieval process provides contextual guidance for the language model without modifying its internal parameters.

D. Retrieval-Augmented Generation Workflow

After semantic retrieval, the retrieved examples are combined with the user's input to construct the final prompt provided to the language model. The complete processing pipeline is implemented using LangGraph [19], an extension of the LangChain framework [20] for building stateful, graph-structured agent workflows, which coordinates the execution of individual processing stages.

The workflow consists of two primary nodes. The first node performs semantic retrieval from the vector database, while the second node generates the final humanized Persian text using the retrieved context and the original user input. A shared workflow state enables efficient communication between processing stages.

E. Language Model Integration

The proposed system employs the Llama 3.1 8B language model executed locally through Ollama [21]. Instead of applying computationally expensive fine-tuning, the retrieved examples are incorporated into the prompt to exploit the in-context learning capability of the language model [4]. This strategy enables the chatbot to generate fluent Persian text while maintaining the semantic meaning of the original input.

Local deployment also improves data privacy because all processing is performed within the local environment without transmitting user data to external cloud services.

F. Backend and User Interface

FastAPI [22] serves as the communication layer between the user interface and the processing modules. User requests are received through RESTful endpoints, forwarded to the LangGraph workflow, and the generated responses are returned to the interface.

A Persian right-to-left web interface was developed to facilitate user interaction. The interface allows users to enter Persian text, optionally indicate a preferred output register, submit the request, and receive the generated humanized output. This register preference is a user-facing option that biases the prompt sent to the language model; it is not a separate model component, and the system's default behavior – and its primary objective – is to humanize the input while preserving its own original register, not to convert it toward a different one. The interface was designed with simplicity and usability in mind while maintaining compatibility with the modular backend architecture.

IV. Implementation and Experimental Results

This section presents the implementation details and experimental results of the proposed Persian text humanization chatbot. The objective of the implementation was to verify the effectiveness of the proposed Retrieval-Augmented Generation (RAG) architecture and to evaluate its ability to generate more natural Persian text while preserving the semantic meaning of the original input.

A. Implementation Environment

The proposed system was implemented using Python as the primary programming language. The backend services were developed using FastAPI, while LangGraph was employed to orchestrate the Retrieval-Augmented Generation workflow. ChromaDB [23] was used as the vector database for semantic retrieval, and multilingual sentence embeddings were applied to convert Persian text into dense vector representations [17].

TABLE III
Software Environment of the Proposed System

Characteristic	Quantity
Processor	10th Generation Intel Core i7
RAM memory	16 GB
GPU	No graphics card required
Language model	llama3.1:8b (via Ollama)
Embedding model	Multilingual MiniLM (384 dimensions)

Database	ChromaDB

The language generation component was implemented using the Llama 3.1 8B language model executed locally through Ollama. This local deployment enables the complete processing pipeline to operate without relying on cloud-based AI services, thereby improving user privacy and reducing external dependencies.

Before execution, the Persian parallel dataset was embedded and indexed into ChromaDB. During inference, the system retrieves semantically relevant examples from the vector database and incorporates them into the prompt before invoking the language model.

B. System Implementation

The complete workflow begins when the user submits Persian text through the web interface. The request is received by the FastAPI backend and forwarded to the LangGraph workflow. The retrieval module converts the input into a semantic embedding and searches the ChromaDB vector database for the most relevant examples.

The retrieved examples are incorporated into the prompt together with the original user input. This prompt is then provided to the Llama 3.1 language model, which generates a more natural and human-like Persian version of the text. Finally, the generated response is returned to the user interface and displayed to the user.

Fig. 2 illustrates the LangGraph workflow implemented in the proposed system.

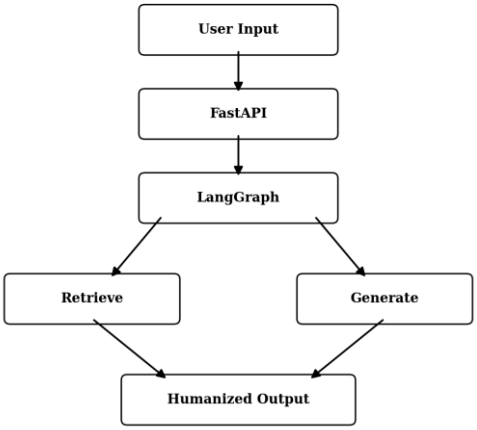


Fig. 2. LangGraph workflow implemented in the proposed system.

C. Experimental Results

Several Persian text samples were processed to evaluate the functionality of the proposed system. The generated outputs indicate that the chatbot is capable of producing more fluent and natural Persian text while preserving the original semantic content.

Fig. 3 shows the user interface of the proposed Persian text humanization chatbot.

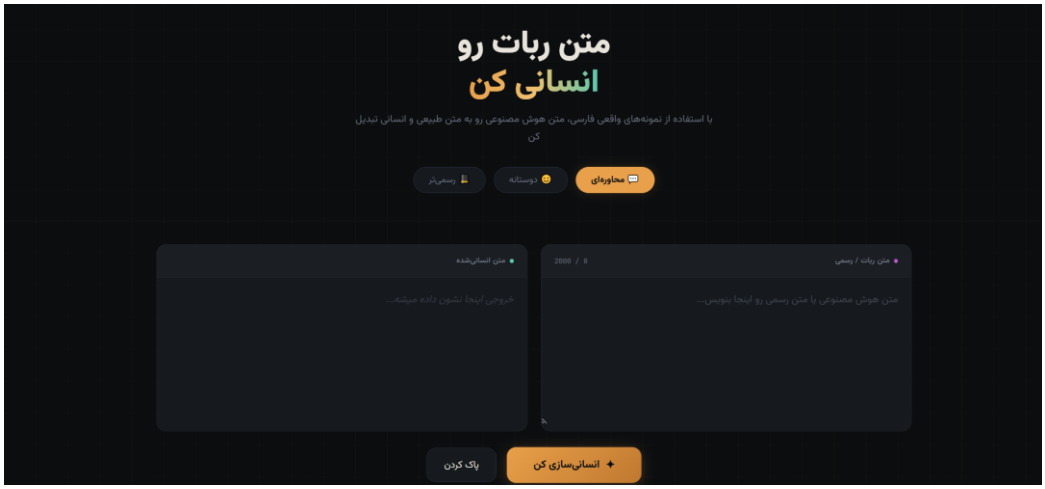


Fig. 3. User interface of the proposed Persian text humanization chatbot.

The rightmost column above labels each example by its real-world context (legal notice, medical instruction, and so on) rather than by the dataset's ten register-metadata categories introduced in Section III-B (casual, simple, friendly, technical, empathetic, persuasive, nostalgic, motivational, calm, humorous); the two label sets are not the same taxonomy and should not be read as interchangeable. These five examples were chosen to illustrate humanization across varied everyday contexts, not to represent the corpus's register distribution, which is characterized separately in Fig. 4(a)–(b).

Representative examples of the generated outputs are presented in Table IV. In every case, the register of the input is preserved – the Legal and Medical examples remain formal, the Friendly example remains casual – while the numeric and factual content (the eight-hour dosing interval, the ten-business-day payment window, the 9:00 meeting time) is carried over exactly. What changes is not the register but the degree to which the phrasing sounds AI-generated: the outputs read as less stiff and more like ordinary human writing in the same register, which is the actual target of humanization as defined in this paper, rather than a shift from one register to another.

TABLE IV

(Sample of AI-Generated Persian Text and Humanized Output)انسانی‌شده خروجی و تولیدشده فارسی متون از نمونه‌ای

زمینه	فارسی انسانی‌شده خروجی	محتوجی هوش توسط تولیدشده فارسی متون
Formal	لطفاً درخواست مرخصی‌تان را تا پایان وقت اداری امروز ثبت کنید.	لطفاً درخواست مرخصی خود را تا پایان ساعت اداری امروز ثبت نمایید.
Medical	بیمار باید دارو را هر هشت ساعت مصرف کند و اگر علائم ادامه داشت، دوباره به پزشک مراجعه کند.	بیمار باید دارو را هر هشت ساعت یک بار مصرف نماید و در صورت ادامه علائم به پزشک مراجعه کند.
Legal	بر اساس مفاد قرارداد، مبلغ باید حداکثر تا ده روز کاری پرداخت شود.	مطابق مفاد قرارداد، پرداخت مبلغ باید حداکثر ظرف ده روز کاری انجام شود.
Business	جلسه فردا ساعت ۹ برگزار می‌شود و حضور همه اعضا ضروری است.	جلسه فردا ساعت ۹:۰۰ برگزار خواهد شد و حضور تمامی اعضا الزامی است.
Friendly	سلام، امیدوارم خوبی. هر وقت فرصت داشتی، لطفاً با من تماس بگیر.	سلام، امیدوارم حالت خوب باشد. اگر وقت داشتی، لطفاً با من تماس بگیر.

A closer inspection of individual examples illustrates the specific linguistic operations performed by the model. In the Legal example, the formal connector مطابق("in accordance with") is replaced with the more conversational بر اساس("based on"), and the bureaucratic nominalized construction انجام شود ("shall be conducted") is collapsed into the direct verb پرداخت شود("shall be paid"); the ten-business-day deadline and the payment obligation itself are unchanged, only the phrasing carrying them is simplified. In the Business example, the model simplifies the over-precise timestamp ۹ ساعت :۰۰ to ساعت

۹without altering the meeting time, replaces the future-tense auxiliary construction برگزار خواهد شدwith the simpler present-tense برگزار می‌شود, and substitutes the legal-register adjective الزامی(“mandatory”) with the softer ضروری(“necessary”) while the attendance requirement itself is preserved.

These edits are consistent across examples: the model favors common near-synonyms over bureaucratic vocabulary, shortens auxiliary verb constructions, and removes redundant notational precision, while leaving the propositional content – facts, numbers, obligations – of each sentence unchanged. We treat this as illustrative evidence of the intended humanization behavior on five hand-picked examples, not as a systematic evaluation; Section IV-F discusses what a systematic version of this analysis would require.

D. Quantitative Evaluation

1) Evaluation Metrics: To quantitatively assess the generated outputs, the ROUGE evaluation metric [24] was employed. ROUGE measures the lexical overlap between the generated text and the corresponding reference text and is widely used for evaluating text generation and text rewriting systems. As noted by Falakaflaki and Shamsfard [6] in the context of Persian formality style transfer, however, overlap-based metrics such as ROUGE and BLEU can be misleading for this class of task: because many Persian words retain an identical surface form across styles and registers, a system that performs little to no rewriting can still obtain a deceptively high overlap score. This work therefore interprets ROUGE in the opposite direction from typical text-generation evaluation – since humanization is expected to substantially reword the input rather than reproduce it, a comparatively low overlap score is the desired outcome, while an unusually high score would instead signal insufficient stylistic transformation. Table V reports the ROUGE scores under this interpretation, and Fig. 4 extends the evaluation with a broader quantitative picture of the dataset, the retrieval module, and the length characteristics of the generated text.

A note on what this evaluation does and does not show is warranted before presenting the numbers. The statistics below characterize the assembled parallel corpus and the retrieval module’s behavior over it – they are not the output of a controlled experiment comparing the full RAG pipeline against a no-retrieval baseline. Specifically, the comparison that would actually justify a claim like “RAG improves humanization” has three parts, none of which has yet been run: (1) the same Llama 3.1 8B model prompted without any retrieved examples, as a baseline; (2) the full RAG pipeline as deployed; and (3) a comparison across retrieval depths, such as top-1 versus the deployed default of top-4, to check whether the amount of retrieved context matters. In addition, it has not been confirmed whether the queries used to compute the retrieval statistics below were held out from the indexed corpus – if a query’s nearest neighbor in ChromaDB is itself (because the same example was both indexed and queried), the reported retrieval precision would be optimistic relative to a genuinely unseen input. Both gaps are treated in full in Section IV-F, where they are listed as required experiments, not as results this paper claims to have. Until they are run, the findings below should be read as a characterization of the system’s components on the assembled data, not as evidence that retrieval improves the end-to-end humanization result.

2) Results: Consistent with the low-overlap-as-success interpretation established above, all five ROUGE variants fall well below 0.02, indicating that the generated outputs share very little surface-level wording or sequence structure with their AI-generated inputs. This confirms that the humanization process performs substantial lexical and structural rewriting rather than trivially reproducing the input, which would have produced ROUGE scores near 1.0. The lower ROUGE-2 F1 score relative to ROUGE-1 F1 further shows that even short, two-word phrases are rarely preserved verbatim, indicating that rewriting occurs at the phrase level and not merely through isolated word substitutions.

TABLE V
ROUGE Evaluation Results

Standard	Qty.	Interpretation
ROUGE-1 Precision	0.019	New terms have been substituted
ROUGE-1 Recall	0.019	Content modified, not copied
ROUGE-1 F1	0.018	Actual conversion performed
ROUGE-2 F1	0.012	Bigram phrasing substantially rewritten
ROUGE-L F1	0.018	Sentence structure changed

Fig. 4(a) and (b) characterize the composition of the Persian parallel dataset introduced in Section III-B.

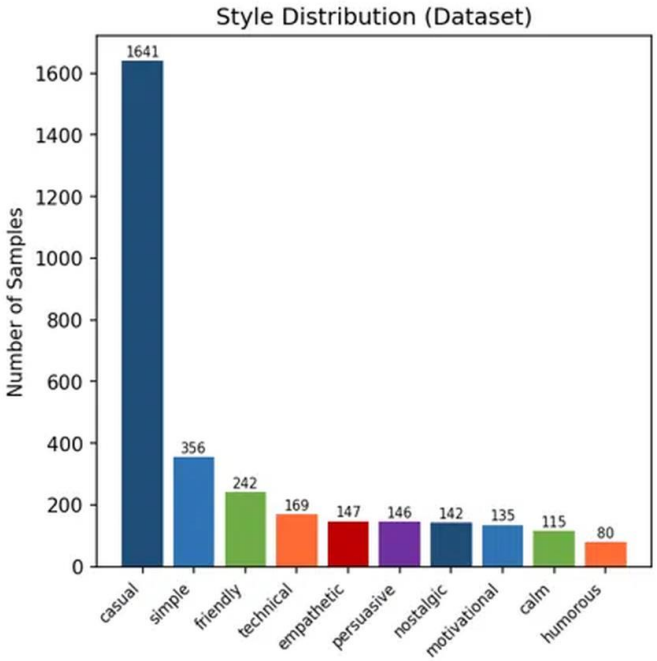


Fig. 4(a). Style distribution of the Persian parallel dataset – number of samples per register category.

The dataset totals 3,173 example pairs distributed across ten register categories, with casual accounting for the majority of samples (1,641) and the remaining registers – simple, friendly, technical, empathetic, persuasive, nostalgic, motivational, calm, and humorous – forming a long tail of 80 to 356 samples each. This distribution describes the composition of the source material collected for the corpus, not a design target of the system itself – the system's objective is natural, humanized output appropriate to each input's own register, not conversion toward casual phrasing specifically. The pronounced skew toward casual is a class imbalance the paper identifies as a limitation, to be addressed by the dataset-expansion direction discussed in Section V.

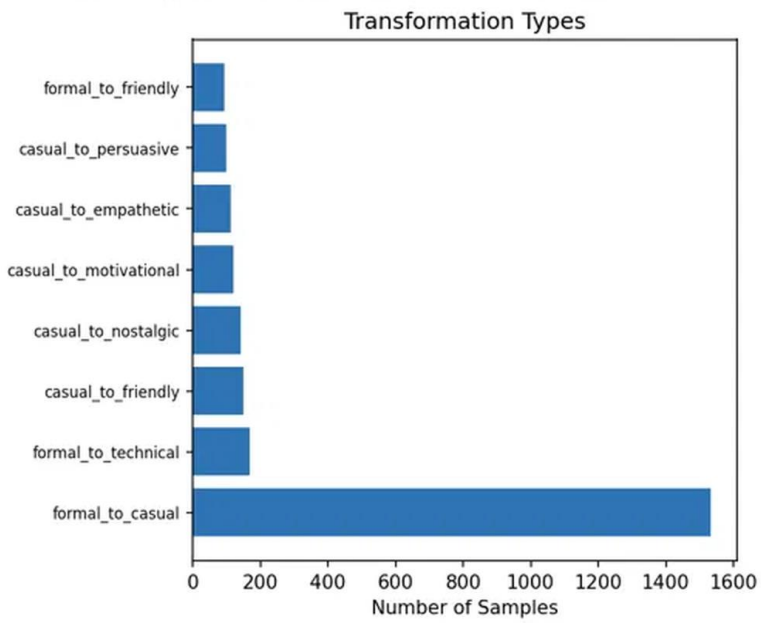


Fig. 4(b). Frequency of the eight recorded register-transition types in the dataset.

Panel (b) shows the register-transition breakdown that mirrors panel (a): formal_to_casual dominates the dataset, while the remaining transitions – e.g., formal_to_technical, casual_to_friendly, casual_to_nostalgic – are comparatively underrepresented. As with panel (a), this reflects the composition of the source material collected for the corpus rather than a target distribution the system is designed to produce. The same class imbalance flagged above applies here.

Panel (c) evaluates the semantic retrieval module described in Section III using Precision@k and Recall@k, computed as the proportion of the top-k retrieved examples that carry the same register metadata as the query input.

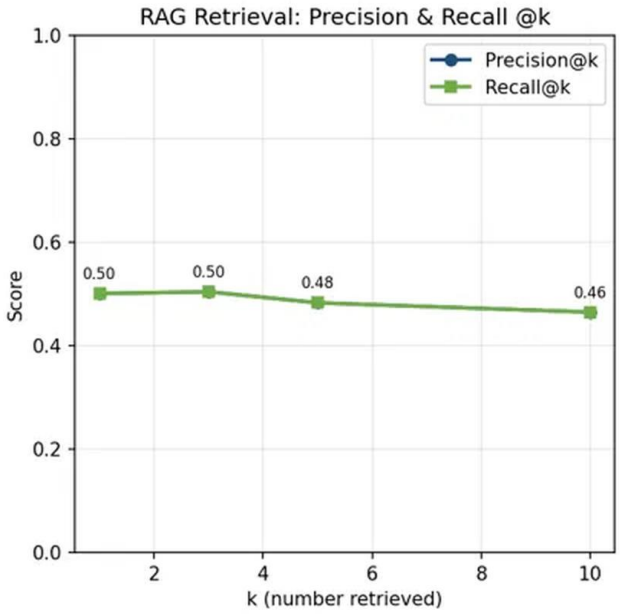


Fig. 4(c). RAG retrieval Precision@k and Recall@k for $k \in \{1, 3, 5, 10\}$.

This is a proxy signal, not a direct measure of retrieval usefulness – two examples can be genuinely useful in-context demonstrations without sharing a register label, and conversely a shared register label does not guarantee the retrieved example is otherwise relevant. It has also not been confirmed that the query set used here excludes examples also present in the indexed corpus, so the absolute values below should be treated cautiously, since self-retrieval has not been conclusively ruled out. With that caveat, both metrics start at approximately 0.50 for k = 1 and k = 3 and degrade gracefully to 0.48 and 0.46 at k = 5 and k = 10, respectively, indicating that roughly half of the examples retrieved from ChromaDB share the input's register at low k, with a modest further decline at higher k consistent with drawing a broader candidate pool from an imbalanced corpus.

Panel (d) compares the word-count distributions of the AI-generated inputs and their humanized counterparts.

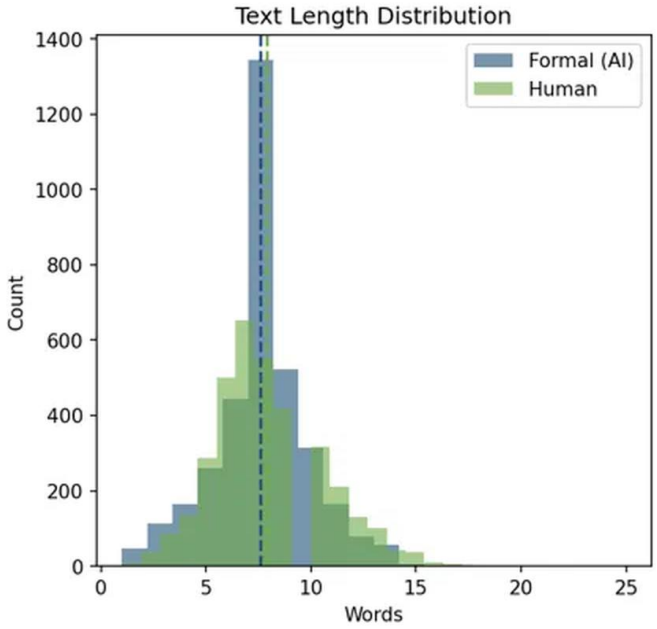


Fig. 4(d). Word-count distribution comparing AI-generated (formal) inputs and humanized outputs.

Both distributions are unimodal and centered around 6–8 words, with substantial overlap, indicating that the humanization process does not systematically truncate or pad the input – a desirable property, since drastic length changes would suggest information loss or unwarranted elaboration rather than faithful rewriting.

Panel (e) presents the ROUGE evaluation metrics from Table V in chart form, between the AI-generated inputs and the human reference text.

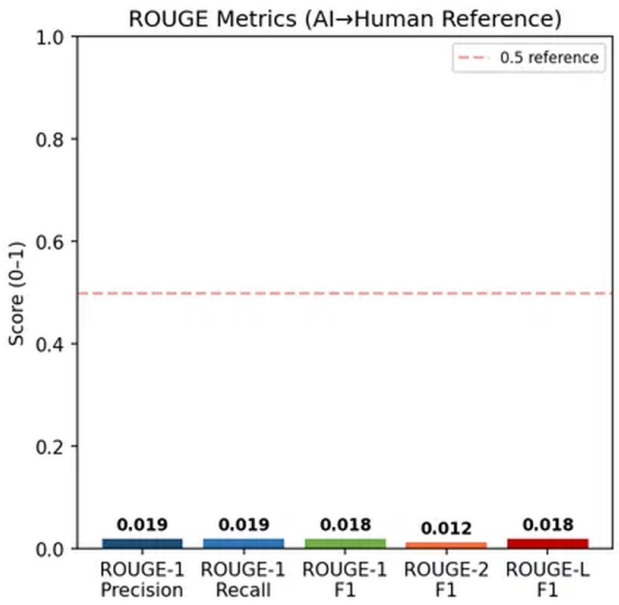


Fig. 4(e). ROUGE evaluation metrics (AI-generated input → human reference).

Because the task is humanization rather than reproduction, and substantial rewriting is expected, a low lexical-overlap score here is interpreted as evidence of successful rewriting rather than of failure – the reverse of how ROUGE is typically read for text-generation tasks. All five ROUGE variants sit well below the 0.5 reference line shown in the chart, consistent with Table V and the discussion above: this indicates substantial lexical and structural rewriting rather than trivial reproduction of the input.

Panel (f) refines the length comparison via the output-to-input length ratio.

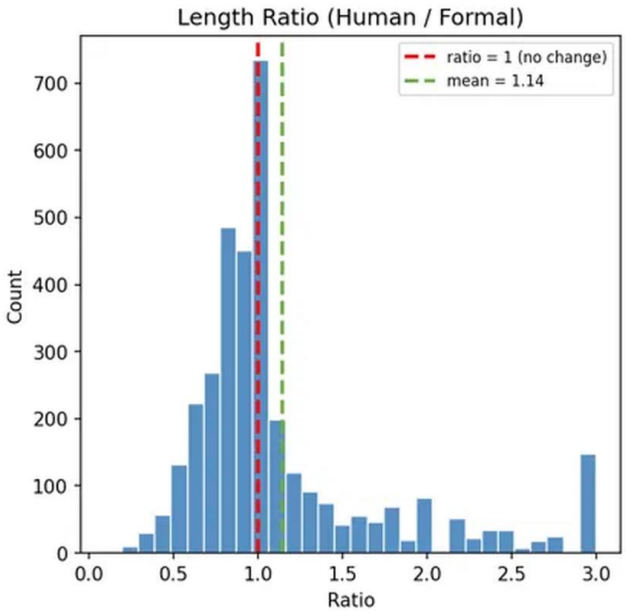


Fig. 4(f). Distribution of output-to-input length ratios (humanized output / AI-generated input).

The ratio peaks close to 1.0, with a mean of approximately 1.14: humanized outputs are, on average, only marginally longer than their AI-generated counterparts. A secondary, smaller mode near a ratio of 3.0 corresponds to a minority of cases – typically more elaborative rewrites – where the model expands substantially on the original input.

Taken together with the ROUGE results in Table V and panel (e), these analyses show that the retrieval module returns register-consistent examples with moderate, not high, reliability, and that the deployed pipeline produces humanized Persian text that remains close in length to the original input while diverging lexically enough to indicate genuine rewriting rather than superficial paraphrasing. As emphasized above, these are properties of the assembled corpus and retrieval module, not evidence – on their own – that retrieval improves generation quality relative to the LLM alone; that comparison is the required next experiment (Section IV-F).

E. Discussion

The pattern in the qualitative examples (Section IV-C) is worth dwelling on, because it suggests why retrieval-augmented prompting might work for this task even without a controlled comparison yet in hand. The edits the model makes – collapsing nominalized bureaucratic constructions into direct verbs, softening AI-typical hyper-formal markers, simplifying auxiliary verb chains – are exactly the kind of local, example-driven substitutions that in-context examples are well suited to teach a language model, as opposed to structural or semantic changes that would more plausibly require parameter updates. This is consistent with the broader in-context learning literature [4], which finds that LLMs are comparatively good at pattern-matching a demonstrated transformation and comparatively worse at generalizing far beyond it. If that account is right, retrieval-augmented prompting should be well matched to within-register naturalization specifically – rewriting stiff phrasing into fluent phrasing without changing the register – and less reliable for humanization tasks that require restructuring meaning rather than re-lexicalizing it – idiomatic expressions and culturally loaded phrasing, the exact failure mode Falakaflaki and Shamsfard [6] report for their own, fine-tuned model, would be a reasonable place to expect this approach to struggle too, since no retrieved example can substitute for genuine cultural or pragmatic understanding.

The retrieval-quality curve (Fig. 4c) offers a second, more cautionary reading. Precision@k and Recall@k hover around 0.50 even at k = 1, meaning that in roughly half of cases, the single most similar example in the corpus does not carry the query's register label. Given that casual and formal_to_casual together dominate the corpus (Fig. 4a–b), this is most plausibly a symptom of the class imbalance documented there rather than a weakness of the embedding model itself: for underrepresented registers such as humorous or calm, the nearest neighbors in a mostly-casual corpus are disproportionately likely to be casual examples that happen to be semantically close in content but carry a different register label. This distinction matters for where future effort should go – it argues for a larger, more balanced corpus before it argues for a different embedding model or a reranking stage, and it is a reminder that register-label agreement is a proxy for retrieval quality, not a definition of it.

Neither of these interpretations, however, is something this evaluation can confirm. Both are offered as explanations consistent with the observed patterns, not as conclusions the current evidence establishes; distinguishing them from alternative explanations is exactly what the baseline- and human-evaluation studies proposed in Section IV-F below are designed to do.

Beyond the evaluation itself, the practical case for the retrieval-based approach rests on what it avoids: no fine-tuning run, no annotated training set beyond the retrieval corpus itself, and a fully offline deployment (Ollama, ChromaDB, and the embedding model all run locally) that keeps user text off any external server. That offline property is a genuine advantage independent of how the quality comparison against fine-tuning eventually turns out, since it is a deployment-cost and privacy argument rather than a quality argument.

F. Limitations

We see the following as the paper's most significant limitations, in roughly descending order of how much they constrain the conclusions that can currently be drawn:

- No baseline comparison. The paper does not currently include the controlled comparison its central claim requires: (1) the Llama 3.1 8B model prompted with no retrieved examples, as a baseline; (2) the full LLM+RAG pipeline as deployed; and (3) a comparison across retrieval depths, such as top-1 versus the deployed default of top-4, to test whether the amount of retrieved context affects output quality. Every claim in this paper about what retrieval contributes is currently supported only by the qualitative pattern in Section IV-C and the architectural reasoning in the Discussion above, not by this comparison. This is the single most important gap and the first item in our future work.
- Retrieval evaluation does not confirm exclusion of self-retrieval. The Precision@k/Recall@k statistics in Fig. 4(c) were not confirmed to use a query set held out from the indexed corpus. If any queried example is also indexed in ChromaDB, it can retrieve itself as a trivial, artificially perfect match, which would inflate the reported precision relative to a genuinely unseen input. This must be corrected – by re-running the evaluation with an explicit held-out split – before these numbers can be interpreted as a measure of generalization.
- No style filtering or re-ranking is implemented. The retrieval module (Section III) performs a single embedding-similarity search and returns the top-k nearest neighbors; it does not filter candidates by register metadata and does not apply a separate re-ranking model. Any register agreement observed between a query and its retrieved examples (Fig. 4c) is an emergent side effect of semantic similarity, not the result of an explicit design constraint. We note this explicitly because both capabilities are easy to assume from the RAG literature generally; in the system described here, they are candidate future extensions, not existing components.
- No human evaluation. Naturalness and fluency – arguably the paper's central claims – are currently assessed only through the authors' own qualitative reading of five illustrative examples, not through independent human raters.
- No semantic-preservation metric. ROUGE and the length-ratio statistics indicate that rewriting occurred and that it did not drastically shorten or lengthen the input, but neither directly measures whether the rewritten meaning, facts, or figures match the original; a low-overlap output that also altered a fact or a number would look identical, by these metrics, to a successful humanization.
- Dataset class imbalance. Casual and formal_to_casual dominate the register-metadata distribution of the 3,173-pair corpus (Fig. 4a–b); results and retrieval quality for underrepresented registers such as humorous or calm should be read with correspondingly less confidence.
- Single LLM, single embedding model. All results reflect one specific pairing (Llama 3.1 8B and a multilingual MiniLM embedding model) at a fixed retrieval depth ($k = 4$); neither the generation quality nor the retrieval quality has been tested against alternative model choices or retrieval depths, so it is not yet known how much of the observed behavior is specific to this configuration.
- No comparison against a fine-tuned baseline. The paper argues for retrieval over fine-tuning on cost and deployment grounds, but does not include a quality comparison against a fine-tuned Persian style-transfer model such as Fa-BERT2BERT [6] run on the same data, so the quality side of that trade-off remains unquantified. This comparison would also only be partially informative,

since Fa-BERT2BERT targets formality style transfer specifically rather than the broader humanization objective of this paper.

None of these limitations is fatal to the paper's central contribution – a working, fine-tuning-free architecture and a dataset and evaluation methodology to study it – but they do mean the paper's research claims should currently be read as preliminary and hypothesis-generating rather than confirmatory.

V. Conclusion

This paper set out to answer a narrow question: whether retrieval-augmented, in-context prompting can make AI-generated Persian text read as natural, human-written text without fine-tuning the underlying model, while preserving its meaning and its original register. We built a working system to test that question – a modular RAG pipeline combining multilingual sentence embeddings, ChromaDB, LangGraph, FastAPI, and a locally deployed Llama 3.1 8B model – and a 3,173-pair Persian parallel corpus, register-labeled for analysis, to drive and characterize it.

What we can say with reasonable confidence: the retrieval module returns register-consistent examples in roughly half of top-ranked cases (with the important caveat that self-retrieval has not been ruled out), the generated text diverges substantially in wording from its input (the low-ROUGE result, read the way Falakaflaki and Shamsfard's critique of overlap metrics suggests it should be read [6]), and stays close to it in length. Qualitative inspection of five hand-picked examples shows the kind of edits one would hope for – reduced AI-typical stiffness, simplified constructions, numbers and obligations preserved exactly – without the register of the input being changed.

What we cannot yet say, and have tried not to overstate, is whether retrieval is the reason for any of this rather than the underlying LLM alone, since that comparison has not yet been run (Section IV-F). We see that as the paper's honest bottom line: a functioning, low-cost, privacy-preserving architecture with encouraging preliminary signal, and a clearly scoped set of experiments – not further engineering – standing between here and a confirmed research result.

A. Future Work

The most direct continuation of this work is not a new component but a missing comparison, and we specify it precisely: (1) the Llama 3.1 8B model with retrieval disabled, as a baseline; (2) the same model with the full RAG pipeline, as deployed; and (3) a comparison across retrieval depths, specifically top-1 versus the deployed default of top-4, to check whether retrieving more examples actually helps. Alongside that comparison, a held-out evaluation split – separating the examples used to build the ChromaDB index from the examples used to query it during evaluation – would close the self-retrieval gap noted in Section IV-F. A small human evaluation of naturalness, meaning preservation, and register match, even on a few dozen examples with two or three Persian/Dari-fluent raters, would give the naturalness claims a footing beyond the authors' own reading of five examples.

Beyond that closer-to-term work, several extensions would strengthen the system itself: expanding the dataset with additional registers and domain-specific examples to correct the class imbalance documented in Section IV-C; adding a register-aware filtering step or a re-ranking stage to the retrieval module – neither of which exists in the current single-pass similarity search – once the baseline experiment above establishes whether retrieval quality is actually the bottleneck; adding BLEU [25] and an embedding-based semantic-similarity score alongside ROUGE, to directly measure meaning and fact preservation rather than inferring it from low lexical overlap; and a direct quantitative comparison

against a fine-tuned Persian style-transfer baseline such as Fa-BERT2BERT [6], understanding that its narrower formality-transfer objective makes the comparison only partially informative for the broader humanization task studied here. Longer-term, packaging the system as a hosted service would be a natural next step once the underlying evaluation questions above are resolved.

Data Availability Statement

The Persian parallel dataset developed for this study, along with the implementation of the retrieval-augmented generation pipeline, is not currently publicly released. These resources are available from the corresponding author upon reasonable request to support reproducibility and further research in Persian text humanization.

Competing Interests

The author declares no competing interests.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2024.

[3] Y. Huang and J. X. Huang, "A survey on retrieval-augmented text generation for large language models," arXiv preprint arXiv:2404.10981, 2024.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, and D. Amodei, "Language models are few-shot learners," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[5] E. Reif, D. Ippolito, A. Yuan, A. Coenen, C. Callison-Burch, and J. Wei, "A recipe for arbitrary text style transfer with large language models," in Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022, pp. 837–848.

[6] P. Falakaflaki and M. Shamsfard, "Formality style transfer in Persian," arXiv preprint arXiv:2406.00867, 2024.

[7] V. Tajalli, F. Kalantari, and M. Shamsfard, "Developing an informal-formal Persian corpus," arXiv preprint arXiv:2308.05336, 2023.

[8] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in Proceedings of the 27th International Conference on Neural Information Processing Systems (NIPS), Cambridge, MA, USA, 2014, pp. 3104–3112.

[9] J. Li, R. Jia, H. He, and P. Liang, "Delete, retrieve, generate: A simple approach to sentiment and style transfer," in Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2018, pp. 1865–1874.

[10] S. Prabhumoye, Y. Tsvetkov, R. Salakhutdinov, and A. W. Black, "Style transfer through back-translation," in Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), 2018, pp. 866–876.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, and I. Polosukhin, "Attention is all you need," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008, 2017.

[12] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, and et al., "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.

[13] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). Online: Association for Computational Linguistics, 2020, pp. 6769–6781.

[14] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv preprint arXiv:1901.04085, 2019.

[15] M. S. Rasooli, F. Bakhtyari, F. Shafiei, M. Ravanbakhsh, and C. Callison-Burch, “Automatic standardization of colloquial Persian,” arXiv preprint arXiv:2012.05879, 2020.

[16] M. Farahani, M. Gharachorloo, M. Farahani, and M. Manthouri, “ParsBERT: Transformer-based model for Persian language understanding,” Neural Processing Letters, vol. 53, pp. 3831–3847, 2021.

[17] N. Reimers and I. Gurevych, “Making monolingual sentence embeddings multilingual using knowledge distillation,” in Proceedings of EMNLP 2020, 2020, pp. 4512–4525.

[18] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in Proceedings of EMNLP 2019, 2019, pp. 3982–3992.

[19] LangChain AI, “LangGraph: Build resilient language agents as graphs,” Computer software, 2024, https://github.com/langchain-ai/langgraph.

[20] H. Chase, “LangChain,” Computer software, 2022, https://github.com/langchain-ai/langchain.

[21] Ollama, “Ollama: Run large language models locally,” Computer software, 2023, https://ollama.com.

[22] FastAPI, “FastAPI: Modern, fast web framework for building APIs with Python,” Computer software, 2023, https://fastapi.tiangolo.com.

[23] Chroma, “Chroma: The AI-native open-source embedding database,” Computer software, 2023, https://github.com/chroma-core/chroma.

[24] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Text Summarization Branches Out. Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81.

[25] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), Philadelphia, PA, USA, 2002, pp. 311–318.